Forklift: LLM-based Lifting C to Idiomatic Rust Through C++

1st Yubo Bai
Department of Computer Science
UC Davis
gabbai@ucdavis.edu

2nd Tapti Palit
Department of Computer Science
UC Davis
tpalit@ucdavis.edu

Abstract*—Rust offers memory safety without sacrificing the performance of C, which has motivated a large body of work on automatically transpiling C to Rust. Existing approaches either apply syntax-directed rewrites that preserve C-style memory management or ask a Large Language Model to translate a whole program in a single pass, crossing the wide semantic gap between the two languages all at once. The Rust code these approaches produce is often unidiomatic and retains many raw pointers and other unsafe constructs, and prior work pays little attention to the runtime performance that matters most for systems software. We present Forklift, an LLM-based pipeline that transpiles C to idiomatic Rust incrementally by first lifting it to C++ and only then to Rust. Because C++ offers abstractions analogous to Rust while sharing most of the LLVM compiler backend with C, each pass stays close to C in performance, allowing each incremental stage to control the performance budget. Forklift derives one program property per pass, confirms that the code still compiles and passes its tests, and discards any pass whose overhead exceeds a configurable threshold, which gives developers an explicit way to trade idiomaticity for performance. Across eight benchmarks from six C codebases and three LLM backends, Forklift produced substantially safer and more idiomatic Rust than a single-pass baseline at comparable runtime. With Claude-Sonnet-4.6 as the backend and the same model driving the single-pass baseline, Forklift raised the standard-library type usage rate by $4.5\times$, reduced total unsafe usages by 82.2% and raw pointers by 73.1%, while matching or beating its performance on five of the eight benchmarks.

I. Introduction

Rust [23], [30] has rapidly become one of the most popular memory-safe systems programming languages. Its strong compile-time safety guarantees, together with performance that rivals C, make it an attractive target for systems software. Rewriting existing software in Rust by hand is expensive and error-prone, which has motivated a large body of work on automated C-to-Rust transpilation. Early transpilers rely on syntax-directed rewrites [1], [14], [21], but the code they produce is largely unidiomatic. Such code preserves C-style memory management and pointer usage instead of adopting the safe abstractions that Rust provides [15]. To close this gap, recent approaches increasingly turn to Large Language Models (LLMs) [12], [15], [17], [36], [46].

Most LLM-based transpilation techniques [12], [15], [33], [36] attempt to translate an entire program from C to Rust in a single pass. At best, some of them [17], [46], [48] first translate C to *unsafe* Rust using syntax-directed approaches [1] and then lift that result to safe Rust with an LLM. In either case the model must cross the gap between two very different languages all at once—C places almost no restrictions on the programmer and allows essentially arbitrary memory manipulation, whereas Rust enforces strict ownership, borrowing, and lifetime rules [23].

The single-pass strategy, used by current C-to-Rust transpilers, forces the model to reason about several intertwined factors at once. These include syntax translation, type mapping, ownership derivation, and lifetime rules. When the model gets any one of these factors wrong, the result is a compiler error. Existing approaches feed the raw compiler error trace back to the LLM and ask it to repair the code. Such a trace often contains many errors from different factors mixed together, so a syntactic mistake can occur right beside a semantic ownership violation. This intermixing might further prevent the model from triaging which compiler errors are critical and from concentrating on one error at a time.

The *libcsv* codebase illustrates why this all-at-once reasoning produces unidiomatic code. Its `csv_parser` struct contains, among other fields, the fields `buf`, `pos`, and `len` that the C code uses together to implement a custom vector type, as shown in Figure 1. The same struct also holds a `realloc` callback, implemented as a function pointer that grows the vector. Accurately transpiling this code to Rust first requires recognizing that these fields collectively implement a vector and then mapping them to the Rust `Vec` type. After that, all of the related fields have to be replaced consistently with that single `Vec` decision. The same kind of rewrite is needed for other complex types as well, for example, when deciding whether a C `char*` type should become Rust's owned `String` type or the borrowed `&str` slice. An LLM that performs a one-shot translation must make these decisions for dozens or even hundreds of types and variables across an entire project, without the ability to focus on one aspect of the transpilation at a time. The lack of such focus leads to unidiomatic code, which is exactly what happens with a single-pass LLM-based pipeline's generation that keeps the `realloc` function, as we discuss in §II.

A second and largely overlooked problem is performance. Existing LLM-based approaches focus on correctness and safety and report the number of remaining `unsafe` blocks [15], [17], [36], [46], but they pay very little attention to

runtime performance. Performance is critical for the systems software [18], [28], [31], [41] that is most worth migrating to Rust, such as operating system kernels, networking stacks, and hypervisors. The goals of safety and performance might even be at odds with one another since a program can reach high performance by using `unsafe` raw pointers everywhere, thereby eliminating runtime safety checks entirely. Balancing safety against performance is therefore equally important.

We present Forklift to address both of these problems through incremental transpilation. Forklift builds on two core insights. The first insight is that the independent aspects of transpilation can be handled one at a time. In one pass, for example, Forklift transpiles C raw buffers into arrays or vectors, and in the next pass it handles the choice between `String` and `&str`. After each pass Forklift also confirms that the code still compiles and passes all unit test cases, which shows that the type mapping derived in that pass is consistent.

The second insight is that performing the transpilation through intermediate *passes* keeps performance under control. Forklift watches the overhead that each pass introduces closely, so a pass that adds significant overhead can be retried or skipped altogether, if desired. Instead of transpiling from C to Rust directly, Forklift therefore routes the transpilation through an intermediate language that stays close to C in performance and yet has a smaller semantic gap to Rust.

C++ is a natural candidate for this intermediate language. C++ offers many of the same abstractions as Rust, including owned pointers, shared pointers, owned strings, and string views, and at the same time C and C++ share the same compiler backend and deliver similar performance when compiled with the LLVM toolchain [42], as we show in §VI-3. We note that one option would be to transpile C to unsafe Rust and then recover the higher-level type information incrementally. However, as we demonstrate in §VI-3, using unsafe Rust as the intermediate language produces worse results than using C++. Forklift therefore first *lifts* C code to C++ before transpiling it to Rust. The LLM performs this C-to-C++ lifting incrementally, in a sequence of passes, and each pass derives one important property of the program under question. Forklift checks the performance after each pass to keep the overhead within a configurable bound, which lets the developer control how much overhead they are willing to tolerate.

In summary, we make the following contributions—

- *Forklift*, an LLM-based pipeline that transpiles C to idiomatic Rust by incrementally lifting it through C++ as a performance-preserving intermediate language. Because C++ provides abstractions analogous to Rust when sharing the LLVM backend with C [42] , each lifting pass stays close to C in performance and closes only a small part of the C-to-Rust semantic gap.
- A performance-controlled transpilation strategy that measures the runtime overhead introduced by each pass against a configurable threshold and discards the results of any pass that exceeds it. This gives developers an explicit way to trade idiomaticity for performance, which prior safety-focused pipelines do not measure or control.
- An evaluation on six real-world C libraries spanning eight benchmarks, showing that Forklift produced substantially safer and more idiomatic Rust than a single-pass LLM-based pipeline at comparable runtime. With Claude-Sonnet-4.6 [5] as the backend and the same model driving the single-pass baseline, Forklift raised the standard-library type usage rate by $4.5\times$, reduced total unsafe usages by 82.2% and raw pointers by 73.1%, while matching or beating the performance of the single-pass baseline on five of the eight benchmarks.

II. Motivation

A. Suitability of C++ as an Intermediate Language

Many of Rust's safe abstractions have direct C++ counterparts in the standard library. As Table II shows, many common Rust standard library types—owned strings, borrowed string slices, dynamic arrays, linked lists, hash maps, optional values, and owned and shared heap pointers—have a one-to-one equivalent in the C++ standard library. Similarly, unowned borrows in Rust correspond to ordinary C++ references. The two languages share a similar set of safety-relevant abstractions, which is what makes C++ a natural intermediate between C and Rust.

For C++ to serve as a performance-transparent intermediate, its standard-library abstractions must exhibit runtime parity with their Rust counterparts, so that any overhead measured across a lifting pass is attributable to the transformation rather than to the source-to-intermediate transition. To determine the performance comparability of the two languages on the safety-relevant abstractions, we measured three microbenchmarks that pair each C++ container in Table II with its Rust counterpart. The array benchmark fills and traverses a dynamic array, using `std::vector` in C++ and `Vec` in Rust. The linked-list benchmark pushes elements onto the list and then traverses and deletes them, using `std::list` in C++ and `LinkedList` in Rust. The hashmap benchmark inserts a series of keys, looks them up, and removes them, using `std::unordered_map` in C++ and `HashMap` in Rust. We compile the C++ sources with `clang++-19` and the Rust sources with `rustc` 1.86.0, both at `-O3` using the same LLVM 19 backend. Table I reports the results. C++ and Rust finish within a small percentage of each other on every data structure, and on the two larger structures Rust slightly outperforms C++. The combination of semantic correspondence and performance parity motivates Forklift's C $\rightarrow$ C++ $\rightarrow$ Rust pipeline. Forklift leverages the similar safety abstractions across C++ and Rust and designs *lifting passes* that incrementally lift the C code through idiomatic C++ safe abstractions, before finally transpiling the idiomatic C++ code to Rust, without injecting unpredictable performance changes at any lifting pass.

B. Limitation of Single-Pass Pipelines—An Example

Figure 1 shows a concrete example demonstrating how a direct, single-pass, LLM translation, using the RustAssure [6]

Original C code

```c
struct csv_parser {
    //...more fields
    unsigned char *entry_buf;
    size_t entry_pos;
    size_t entry_size;
    void *(*realloc_func)(void *, size_t);
};
size_t csv_parse(
    struct csv_parser *p,
    //...more parameters
) {
    //...more codes
    if (entry_pos == p->entry_size){
       //use realloc to grow buffer
       realloc_func(p->entry_buf, new_size);
    }
    p->entry_buf[p->entry_pos] = c;
}
```

Single-pass LLM-transpiled Rust code

```rust
pub struct csv_parser {
    //...more fields
    pub entry_buf: *mut u8,
    pub entry_pos: size_t,
    pub entry_size: size_t,
    pub realloc_fn: fn(*mut c_void,
                       size_t)
                    -> *mut c_void,
}
pub unsafe fn csv_parse(
    p: *mut csv_parser,
    //...more parameters
) -> size_t {
    //...more codes
    if (entry_pos == (*p).entry_size) {
        //use realloc to grow buffer
        realloc_fn((*p).entry_buf, new_size);
    }
    *(*p).entry_buf.add((*p).entry_pos) = c;
    // unsafe raw-pointer write
}
```

Fig. 1. Single-pass LLM-based pipeline output for libcsv: the `entry_buf` field remains a raw `*mut u8`, forcing the `unsafe` helper `csv_increase_buffer` to re-allocate via `realloc_fn` and the per-byte write in `csv_parse` to use raw-pointer arithmetic.

TABLE I
RUNTIME COMPARISON BETWEEN C++ AND RUST STANDARD LIBRARY DATA STRUCTURES (TIME IN S).

Data Structure	C++	Rust
Array	17.02	17.25
Linked list	40.76	36.98
Map	46.81	41.02

TABLE II
ANALOGOUS ABSTRACTIONS BETWEEN C++ AND RUST. BOTH LANGUAGES PROVIDE ABSTRACTIONS FOR THE SAME SET OF COMMON DATA STRUCTURE AND OWNERSHIP PATTERNS, WHICH LETS FORKLIFT LIFT EACH C FEATURE INTO ITS C++ FORM AND THEN TO ITS RUST COUNTERPART WITH A SMALL SEMANTIC GAP ON EVERY LIFTING PASS.

Concept	C++	Rust
Owned string	`std::string`	`String`
Borrowed string slice	`std::string_view`	`&str`
Dynamic array	`std::vector`	`Vec`
Linked list	`std::list`	`LinkedList`
Hash map	`std::unordered_map`	`HashMap`
Optional value	`std::optional`	`Option`
Owned heap pointer	`std::unique_ptr`	`Box`
Shared heap pointer	`std::shared_ptr`	`Rc`
Borrowed reference	`T&`	`&T`

pipeline, preserves the C representation verbatim even when Rust offers a cleaner equivalent. The *libcsv* parser maintains a growable byte buffer named `entry_buf` that the routine `csv_parse` fills one byte at a time as it scans the CSV input. In the original C source, `entry_buf` has a plain `char *` type together with two manually tracked fields, `entry_pos` for the next write index and `entry_size` for the current capacity. A helper function `csv_increase_buffer` calls `realloc` to grow the buffer whenever it fills up. When a single-pass LLM-based pipeline translates this code directly in one pass, it carries the same shape into Rust as shown in Figure 1. The field `entry_buf` stays as a raw `*mut u8`, `csv_increase_buffer` remains `unsafe` and still invokes `realloc_fn` to re-allocate the buffer by hand, and each appended byte is written through raw-pointer arithmetic that indexes into the buffer using the manually maintained `entry_pos` cursor. The output never adopts Rust's `Vec` type, which can grow on demand and exposes a `push` method that appends a byte and updates the buffer's length in one call. Switching to `Vec` would eliminate the cursor field, the manual `realloc` call, and the surrounding `unsafe` blocks, producing a far more idiomatic and safer translation.

III. DESIGN

Forklift transpiles the C code through a series of intermediate C++ lifting passes that gradually *lift* it to Rust. As shown in Figure 2, each of these passes corresponds to a particular type-mapping task. After each pass finishes, both the correctness and performance assurance of the transpiled code are verified, as discussed in §III-3.

1) Preprocessing: Forklift begins by preprocessing the C codebase into per-function units and extracting their dependency information that the later lifting passes rely on. It first expands the header files and macros, then splits the source into individual C functions and identifies every function defined in the original code. This process ignores libc functions such as `atof` and `atoi` because Forklift only transpiles the functions that belong to the source codebase. Similarly, any type defined

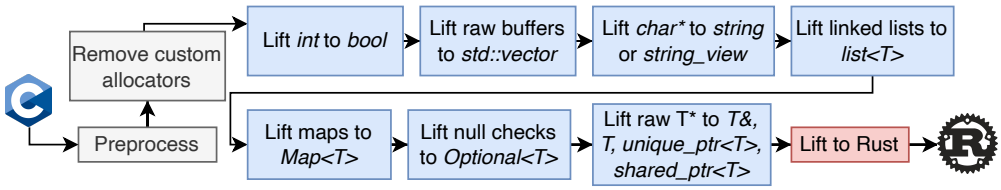

Fig. 2. Overview of the Forklift transpilation pipeline. Forklift preprocesses the input C code and then incrementally lifts it through a sequence of feature-specific LLM-based passes before performing a final lift to Rust.

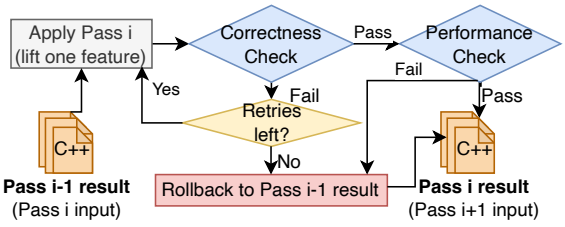

Fig. 3. Per pass correctness and performance check in Forklift. After each lifting pass, Forklift runs the correctness check and the performance check. A pass is kept only if both checks pass, otherwise Forklift rolls back to the previous pass's result before proceeding to the next pass.

in the C standard library, such as `_IO_FILE`, is also ignored. To limit the code that is sent to the LLM for transpilation, Forklift removes any `typedefs` and any `struct` or `enum` declarations that the codebase never uses, but was imported due to C header file inclusions.

Forklift aims to provide the LLM with *maximum relevant context* it needs to transpile a particular code chunk. Therefore, Forklift first extracts a type dependency graph and a function dependency graph, and transpiles the codebase according to these dependency graphs. To construct the type dependency graph, Forklift first extracts four categories of user-defined type definitions, namely, `typedef`, `struct`, `union`, and `enum` declarations. Similar to previous approaches [6], to facilitate the transpilation of generic pointer fields, such as `char*` and `void*`, the type dependency graph also contains the *uses* of each generic pointer field. Forklift constructs a function dependency graph using a statically constructed program control graph. Forklift also extracts the global variables along with their dependent types. Finally, Forklift compiles the original C code and executes it five times to record a performance baseline for the later lifting passes.

2) Multi-pass LLM Invocations: Forklift invokes the LLM to perform multiple *lifting passes* over the C codebase. The details of these LLM passes are described below.

Global Transpilation Rules. Forklift provides a *system prompt* that establishes a persistent context and instructions for the LLM used to transpile the code. Forklift uses this system prompt to inform the LLM of global transpilation rules. The system prompt's overarching goal is that C++ vocabulary types [10] are adopted idiomatically, so the LLM must change how a data structure is *used* and not simply rewrite it syntactically while continuing to manipulate it in "C-style". These C++ types embed information crucial for the final transpilation pass to Rust, such as ownership and nullability, as well as map more closely to Rust types than C types.

The system prompt stays the same across every pass and encodes a small set of rules that steer the model toward idiomatic, ownership-aware code rather than a superficial syntactic port. The first rule governs allocation of C++ objects and mandates the use of constructors and destructors. Any `struct` that carries an owning member, such as an owned `vector`, an owned `string`, a unique or shared owning pointer, or an optional value, and any type that defines its own constructor or destructor, must be created through C++ object construction such as `new T{}` or the standard `std::make_unique` helper, and never through `malloc(sizeof(T))`. The second rule enforces type-change propagation and mandates that when a variable or field's type changes, every read and write that touches that variable or field is updated in the same edit. The third rule requires that ownership transfer be made explicit through an explicit `std::move` rather than left implicit. The fourth rule enforces signature propagation, so a type change must reach every callee declaration, forward declaration, and call site in the same edit. Together these rules keep each edit internally consistent and prevent the model from producing code that merely renames C constructs without committing to C++ ownership semantics.

Forklift Rewrite Chain. Forklift consists of a rewrite chain of the nine lifting passes described in Table III. Before applying a particular pass, Forklift first performs a *suitability check* and asks the LLM if the proposed change will make the code more idiomatic or not. The subsequent pass is applied only if the model confirms affirmatively. This check allows the LLM to introspect and mindfully apply a lifting pass only when needed, instead of mindlessly trying to apply a pass. For example, we observed that asking the LLM to lift custom C map types to the C++ `Map<T>` type caused the LLM to incorrectly adopt the `Map<T>` type where it was not applicable. Adding this *suitability check* prevents such cases from occurring.

Once the suitability check passes, within each pass, Forklift first transpiles types in topological order over the type dependency graph. Each type is translated individually, with its already-translated dependencies supplied as context so the resulting code compiles. When a set of types forms a strongly connected component (SCC), they reference each other and

cannot be ordered. Therefore, the entire SCC is transpiled as a single unit in one LLM request. Similarly, Forklift transpiles functions in topological order over the call graph—each function is translated individually, with the signatures of its callees supplied as context so the call sites resolve and the result compiles. Strongly connected components (SCCs) involving functions are transpiled as a single unit in one LLM request.

Within a pass, Forklift translates one compilation unit at a time in topological order and assembles the outputs incrementally. Each pass keeps a growing buffer of the units already translated in the current pass and compiles each new unit against that buffer together with the shared type-context block, so every function is checked against the already-translated code that it depends on. The buffer left at the end of the pass is the merged source that the next pass consumes. After a pass finishes, Forklift outputs every translated type and function body, so the C++ that pass i produces becomes the input that pass $i{+}1$ reads. The first pass is the only one that consumes the original C code. Each pass includes a compilation check that ensures that the produced output is well formed and repairs failures with a bounded feedback loop.

Forklift adds two mitigations that remove the common sources of inconsistency introduced by the incremental transpilation of functions. First, during the transpilation of a function, say `f1`, the LLM can rewrite the signature of `f2`, on which `f1` depends, by rewriting the call site that invokes `f2`. This rewriting will, however, break all other call sites invoking `f2` in other functions. To mitigate this potential inconsistency, Forklift extracts the signature of every translated function and keeps it as the single source of truth for that function. A caller receives only the stored signatures of its callees rather than the entire function bodies. A second source of inconsistency arises when a lifting pass *undoes* a change implemented by a previous pass. Therefore, Forklift attaches to every pass prompt a ledger of what the other passes are responsible for and instructs that the work of passes that have already run must be preserved unchanged. This ensures consistency and avoids any regressions such as the case where a `struct` field that an earlier pass turned into an owned string being reverted to a character pointer. Similarly, the prompt instructs that the work of passes that have not run yet must be deferred rather than done opportunistically. Together, these mitigations keep each pass's edits consistent across function boundaries and non-destructive of the passes around it.

3) Correctness and Performance Assurance: To ensure functional correctness and performance parity, the generated code undergoes a correctness and performance check at the end of each pass, as Figure 3 illustrates. After finishing each pass's transpilation, Forklift runs the unit test cases, as well as any end-to-end test cases in the original codebase to guarantee the functional correctness for each single pass. If the correctness check fails, Forklift will execute a repair loop and re-run the whole pass with the error information.

The performance check operates on a threshold τ that bounds the acceptable runtime degradation introduced by a single pass's transformations. After Pass i finishes, Forklift runs the codebase's performance harness five times and compares the mean elapsed time t_i against the previous pass's mean t_{i-1}; the pass is judged to have regressed when $t_i > t_{i-1} \cdot (1 + \tau)$. Forklift exposes the performance check threshold τ as a configurable parameter that defaults to 5%. Forklift accepts Pass i's result only when both the correctness and performance checks pass, rolling back to Pass $i-1$'s result otherwise.

IV. Implementation

Forklift interfaces with the LLM backends through Python. It compiles every C source and C++ intermediate with Clang 19 and Clang++ 19, and every Rust output with Rustc 1.86.0, all at the `-O3` optimization level. The system averages each reported runtime over five runs on the same machine. Forklift performs the static analysis in its preprocessing stage with a sequence of Clang tools built on Clang's AST matchers [3]. These tools expand every header file and macro into a per-function `.i` file, remove the typedef, struct, and enum definitions that the root function never reaches, and extract the field-use patterns of the generic `void *` and `char *` pointers inside the remaining structs. Forklift uses the tree-sitter [7] incremental parser to extract function signature information from the translated C, C++, and Rust code, which it then propagates to each function's callers across passes.

V. Evaluation

We evaluate Forklift on six C applications and libraries—*libcsv*, *cjson*, *libbmp*, *pagerank*, *skiplist*, and *urlparser*. We evaluate *cjson* and *libbmp* with two API functions, thus together yielding eight benchmarks. For both *cjson* and *libbmp* we exercise two APIs each, one that parses and one that writes. These applications are commonly used as benchmarks in previous C-to-Rust transpilation work [6], [13], [15], [17], [21], [46], [48]. Table IV lists the eight C benchmarks together with the workload we use to measure each one's runtime. The six codebases span 288 to 1,222 lines of C, with *libcsv* at 688, *cjson* at 1,222, *libbmp* at 546, *pagerank* at 288, *skiplist* at 656, and *urlparser* at 736 lines.

We compare Forklift against a traditional LLM-based pipeline that transpiles each codebase in a *single pass*, prompting the LLM to directly generate Rust code from C. We build this baseline by adapting the RustAssure [6] pipeline to use the Claude-Sonnet-4.6 as its backend, so that the comparison isolates the effect of Forklift's multi-pass lifting from the choice of LLM. We select three LLM models as Forklift's backends—Anthropic's Claude-Opus-4.6 and Claude-Sonnet-4.6, and OpenAI's GPT-5.4 [4], [5], [34].

To illustrate the performance vs. safety tradeoff, we evaluate Forklift with four values of the per-pass performance-degradation threshold τ—5%, 15%, 30%, and ∞ for the Claude-Sonnet-4.6. A lower value of τ signals a higher priority for performance, with the τ value of ∞ indicating that we are prioritizing safety over performance. The Claude-Opus-4.6 and GPT-5.4 backends run at $\tau = \infty$ only.

TABLE III
PER PASS PROMPT STRUCTURE. EACH PASS TARGETS ONE LANGUAGE FEATURE.

Pass	Language Feature	C Source Construct	Target C++ Idiom	Additional Prompt Hints
1	Memory Management	custom allocator	None	None.
2	Boolean	`int` flags used as True or False value	`bool`	None.
3	Array	`T*` buffer	`std::vector<T>`	Use `std::vector`'s built-in member functions (`push_back`, `resize`, `size`, iterators) instead of manually tracking a write cursor or keeping the original `malloc` / `realloc` / `free`.
4	String	`char*` / `const char*` / `unsigned char*`	`std::string` (owner) or `std::string_view` (view)	Rewrite owner pointers as `std::string`; rewrite non-owning aliases as `std::string_view`.
5	Linked List	Linked list	`std::list<T>`	None.
6	Map	Hash table	`std::unordered_map<K,V>`	None.
7	Nullable value	`nullptr` with a NULL check	`std::optional<T>`	Wrap variables in `std::optional<T>` when they may be null and the surrounding control flow performs explicit null checks on them.
8	Ownership	Raw `T*`	`T&` (reference), `T` (value), `std::unique_ptr<T>`, `std::shared_ptr<T>`	Rewrite a raw pointer as `std::unique_ptr<T>` when it owns the object; as `T&` when it only dereferences without transferring ownership.
9	Transpile C++ to idiomatic Rust Code.			

TABLE IV
THE BENCHMARKS AND THE WORKLOAD USED TO MEASURE THEIR RUNTIME. CJSON AND LIBBMP EACH CONTRIBUTE TWO API LEVEL BENCHMARKS—A PARSE AND A WRITE OVER THE SAME LIBRARY.

Benchmark	Functionality	Performance workload
libcsv	CSV parsing and validation library	Validating 1.6 GB CSV (∼1.06M rows, 19 cols)
cjson (parse)	JSON parsing and generation library	5,000 iterations parsing an 809-entry JSON
cjson (write)		8,000 iterations building and serializing an 8,000-key JSON object
libbmp (parse)	Bitmap (BMP) image library	Parsing 10,000 synthetic 500×500 bitmaps
libbmp (write)		Writing one 8000×8000 bitmap
pagerank	PageRank graph algorithm	LiveJournal graph (∼4M nodes, ∼35M edges) run to convergence
skiplist	SkipList implementation	1,000,000 key insertions and lookups
urlparser	URL parsing library	Parsing 10,000 URLs over 1,000 iterations

A. Standard Library Type Usages

An important metric of idiomaticity for userspace Rust applications is the use of standard library types for common data structures instead of mimicking the C implementations for the same. We compute the standard-library type usage rate by collecting the types that appear in user-defined type definitions, function parameters, and return types. For user-defined types such as `struct` and `enum`, we recursively inspect every field so that nested types are recorded as well. We then classify each recorded type as either a Rust standard-library construct—e.g., `Vec`, `String`, `Box`, `Rc`, `Arc`, `Option`, `Result`, `HashMap`, `HashSet`, or a slice/reference (`&T`, `&str`, `&[T]`)—or a non-standard-library construct, such as a custom user-defined type or a raw pointer. Let N_{std} and N_{raw} denote the respective counts. The standard library usage rate is computed as $R_{\text{std}} = N_{\text{std}}/(N_{\text{std}} + N_{\text{raw}})$.

Table V reports R_{std} per benchmark and method. Averaged across the eight benchmarks, the four main methods rank as follows. Claude-Sonnet-4.6 at the most permissive threshold $\tau = \infty$ leads at 84.9%, GPT-5.4 follows at 83.4%, Claude-Opus-4.6 at 82.2%, and the single-pass LLM-based pipeline (SPP) trails at 18.8%. Claude-Sonnet-4.6 at $\tau = \infty$ is the strongest configuration across all the models Forklift has tested. Since the single-pass baseline also runs on Claude-Sonnet-4.6, comparing it against Forklift's Claude-Sonnet-4.6 backend at $\tau = \infty$ holds the model fixed and isolates the effect of the multi-pass lifting alone, which raises R_{std} from the baseline's 18.8% to 84.9%, for a $4.5\times$ improvement.

Within the Claude-Sonnet-4.6 model, the four thresholds rank monotonically with τ. $\tau = 5\%$ averages 52.2%, $\tau = 15\%$ averages 61.4%, $\tau = 30\%$ averages 72.0%, and $\tau = \infty$ averages 84.9%. This indicates that increasing the performance budget allows the models to generate more idiomatic Rust code using the standard library types.

B. Unsafe Rust Usages

Unsafe Rust allows the programmer to opt out of Rust's safety guarantees. Therefore, the usage of unsafe Rust must be minimized. To quantify the amount of unsafe Rust used, we report two common metrics [6], [48] for every translated Rust output. *Total unsafe usages* include every unsafe Rust statement in the transpiled code. We count code implementing unsafe traits, unsafe function calls, raw-pointer dereferences,

TABLE V
STANDARD LIBRARY TYPE USAGE RATE R_{STD} (%) PER BENCHMARK AND METHOD. **SPP** IS THE SINGLE-PASS LLM-BASED PIPELINE BASELINE, ADAPTED FROM THE RUSTASSURE ARTIFACT [6].

	Forklift						
Benchmark	Opus	GPT-5	Sn-5%	Sn-15%	Sn-30%	Sn-∞	SPP
libcsv	**84.4**	81.2	81.8	82.2	77.8	76.1	33.3
cjson (parse)	76.0	81.7	10.4	32.4	83.3	**83.8**	4.7
cjson (write)	70.4	80.0	9.1	32.9	**82.0**	75.7	4.4
libbmp (parse)	**100.0**	94.1	**100.0**	**100.0**	**100.0**	**100.0**	29.4
libbmp (write)	**100.0**	92.9	75.9	**100.0**	**100.0**	**100.0**	26.3
pagerank	87.5	**100.0**	**100.0**	12.5	12.5	**100.0**	25.0
skiplist	39.1	41.3	14.0	38.5	38.5	**43.5**	17.5
urlparser	**100.0**	95.9	26.4	92.7	81.6	**100.0**	9.6

TABLE VI
TOTAL UNSAFE USAGES PER BENCHMARK AND METHOD. **SPP** IS THE SINGLE-PASS LLM-BASED PIPELINE BASELINE.

	Forklift						
Benchmark	Opus	GPT-5	Sn-5%	Sn-15%	Sn-30%	Sn-∞	SPP
libcsv	**10**	14	64	35	42	36	217
cjson (parse)	96	47	373	284	52	**21**	508
cjson (write)	96	47	412	296	**44**	73	504
libbmp (parse)	5	11	**3**	5	5	**3**	47
libbmp (write)	5	16	40	7	7	**4**	48
pagerank	1	**0**	**0**	25	25	3	24
skiplist	70	56	224	**48**	63	73	183
urlparser	177	**2**	180	100	41	98	213

TABLE VII
NUMBER OF UNSAFE RAW-POINTER USAGES PER BENCHMARK AND METHOD. **SPP** IS THE SINGLE-PASS LLM-BASED PIPELINE BASELINE.

	Forklift						
Benchmark	Opus	GPT-5	Sn-5%	Sn-15%	Sn-30%	Sn-∞	SPP
libcsv	**9**	12	11	10	12	13	36
cjson (parse)	21	19	66	48	19	**13**	73
cjson (write)	26	**14**	67	48	18	30	78
libbmp (parse)	**0**	2	**0**	**0**	**0**	**0**	14
libbmp (write)	**0**	3	9	**0**	**0**	**0**	11
pagerank	1	**0**	**0**	7	7	2	6
skiplist	20	20	39	**14**	23	20	35
urlparser	5	**0**	45	23	14	6	60

`static mut` accesses, and union field accesses [2] Additionally, we also report the number of *unsafe pointers*, `*const T` and `*mut T`, used in each transpiled codebase.

Tables VI and VII report the two metrics across the eight benchmarks. Averaged across benchmarks, GPT-5.4 has the lowest mean on both, followed by Claude-Sonnet-4.6 at $\tau = \infty$ and Claude-Opus-4.6, with the single-pass LLM-based pipeline last. For total unsafe usages, the means are GPT-5.4 at 24.1, Claude-Sonnet-4.6 at $\tau = \infty$ at 38.9, Claude-Opus-4.6 at 57.5, and the single-pass LLM-based pipeline at 218.0. For unsafe pointers, GPT-5.4 again has the lowest mean at 8.8, Claude-Opus-4.6 and Claude-Sonnet-4.6 at $\tau = \infty$ are nearly tied at 10.3 and 10.5, and the single-pass LLM-based pipeline is far higher at 39.1. Across all models, GPT-5.4 is the strongest on both metrics, reducing total unsafe usages by 88.9% and unsafe pointers by 77.5% relative to the single-pass LLM-based pipeline. Since that baseline also runs on Claude-Sonnet-4.6, comparing it against Forklift's Claude-Sonnet-4.6 backend at $\tau = \infty$ holds the model fixed and isolates the effect of the multi-pass lifting alone, which reduces total unsafe usages by 82.2% and unsafe pointers by 73.1%.

Within the Claude-Sonnet-4.6 model, looser τ thresholds again produce more idiomatic Rust because the performance check stops rolling back the passes that eliminate unsafe usage. For unsafe pointers, the trend is monotonic with τ. $\tau = 5\%$ averages 29.6 pointers, $\tau = 15\%$ averages 18.8, $\tau = 30\%$ averages 11.6, and $\tau = \infty$ averages 10.5. Total unsafe usages follow the same broad downward trend, with the mean falling from 162.0 at $\tau = 5\%$ to 100.0 at $\tau = 15\%$ and 34.9 at $\tau = 30\%$, although $\tau = \infty$ at 38.9 climbs back above $\tau = 30\%$ rather than continuing to fall.

C. Performance

As shown in Table VIII, Forklift's Claude-Sonnet-4.6 backend with a $\tau = \infty$ performance threshold produces transpilations that are at least as fast as the Claude-Sonnet-4.6-based single-pass pipeline for 5 out of 8 benchmarks. The remaining 3 benchmarks—*skiplist*, *cjson* (parse), and *cjson* (write)—incur 7%, 28%, and 38% overhead, respectively. Importantly, for Forklift, these competitive results come with significantly higher safety guarantees, as shown in Table VI. Across 8 benchmarks for the final runs, the average standard deviation and coefficient of variation are 207.10 ms and 3.47%, which indicates that the measured runtimes are stable.

Within the individual models, for *libcsv*, Claude-Opus-4.6 runs in 6.2 s, GPT-5.4 in 6.1 s, and Claude-Sonnet-4.6 in 6.0 s, all faster than the single-pass LLM-based pipeline's 7.0 s. For *cjson* (parse), the single-pass LLM-based pipeline leads at 7.4 s, ahead of GPT-5.4 at 8.8 s, Claude-Sonnet-4.6 at 9.5 s, and Claude-Opus-4.6 at 10.0 s. For *cjson* (write), the single-pass LLM-based pipeline, SPP, is the fastest at 3.4 s, followed by Claude-Opus-4.6 at 4.5 s, Claude-Sonnet-4.6 at 4.7 s, and GPT-5.4 at 5.7 s. For *libbmp* (parse) and *libbmp* (write), every Forklift result either matches or beats the single-pass LLM-based pipeline (7.1 and 4.8 s respectively). For *pagerank*, Claude-Sonnet-4.6 matches the single-pass pipeline at 2.8 s, with Claude-Opus-4.6 and GPT-5.4 at 3.7 s. For *skiplist*, every Forklift result trails the single-pass LLM-based pipeline (8.0 s) by 0.6 to 1.1 s. For *urlparser*, Claude-Sonnet-4.6 leads at 2.9 s, ahead of Claude-Opus-4.6 and the single-pass LLM-based pipeline (both at 3.4 s), while GPT-5.4 takes 4.9 s.

While the threshold τ has a consistent relationship to safety, Table VIII shows that a similar correlation for performance holds only for 6 out of 8 benchmarks. While the $\tau = 5\%$

TABLE VIII
PERFORMANCE OF THE RUST OUTPUT (IN SECONDS) PER BENCHMARK AND METHOD, WITH THE PER-BENCHMARK MEDIAN C BASELINE. **SPP** IS THE SINGLE-PASS LLM-BASED PIPELINE BASELINE.

		Forklift						
Benchmark	**C base**	Opus	GPT-5	Sn-5%	Sn-15%	Sn-30%	Sn-∞	SPP
libcsv	8.2	6.2	6.1	**5.9**	6.5	6.7	6.0	7.0
cjson (parse)	7.9	10.0	8.8	9.0	10.5	9.7	9.5	**7.4**
cjson (write)	3.7	4.5	5.7	3.7	4.4	4.3	4.7	**3.4**
libbmp (parse)	6.2	6.5	6.8	**6.0**	6.7	**6.0**	6.1	7.1
libbmp (write)	4.9	4.7	4.7	**4.3**	5.2	4.7	4.8	4.8
pagerank	2.8	3.7	3.7	3.3	**2.7**	3.0	2.8	2.8
skiplist	7.8	8.7	9.1	**7.8**	8.7	8.9	8.6	8.0
urlparser	3.4	3.4	4.9	3.3	3.3	4.0	**2.9**	3.4

configuration has the overall best runtime performance, for the *pagerank* and *urlparser* benchmarks, $\tau = 5\%$ has slower performance than $\tau = \infty$. Moreover, for the $\tau = 15\%$ and $\tau = 30\%$ configurations, we do not observe a direct correlation between the performance threshold and the final runtime overhead.

D. Token Cost

Table IX reports the total token consumption per benchmark (input plus output combined) for each Forklift backend (running at $\tau = \infty$) and the single-pass LLM-based pipeline. The last three rows of the table sum the tokens, the API cost in US dollars, and the per-benchmark average cost across the eight benchmarks. We compute cost from the published per-million-token API prices for each model, with Claude-Opus-4.6 charging \$5 per million input tokens and \$25 per million output tokens, Claude-Sonnet-4.6 charging \$3 and \$15, and GPT-5.4 charging \$2.50 and \$15.

The three Forklift backends rank differently on tokens consumed and on cost incurred. By total tokens, GPT-5.4 consumes the most at 6.58 M, followed by Claude-Opus-4.6 at 5.31 M and Claude-Sonnet-4.6 at 5.15 M, with the single-pass LLM-based pipeline at only 0.31 M. By cost, Claude-Opus-4.6 is the most expensive at \$39.19 because its per-token prices are roughly twice those of the other two backends, followed by GPT-5.4 at \$22.16, Claude-Sonnet-4.6 at \$20.88, and the single-pass LLM-based pipeline at \$2.01. Forklift's pipeline spends $10\times$ to $19\times$ more than the single-pass LLM-based pipeline in absolute API cost because it issues nine pass-by-pass calls per benchmark rather than a single end-to-end translation.

Despite the higher absolute cost, Forklift's API spend stays manageable in practice. The most expensive configuration, Claude-Opus-4.6, averages about \$5 per benchmark across all eight benchmarks. The cheaper configurations, Claude-Sonnet-4.6 and GPT-5.4, average roughly \$2.60 and \$2.80 per benchmark. Lifting an entire codebase to idiomatic and safer Rust for under \$6 in API cost is a small fraction of the engineering time the lift would otherwise consume by hand.

TABLE IX
PER-BENCHMARK TOTAL TOKEN CONSUMPTION (INPUT PLUS OUTPUT COMBINED, IN THOUSANDS OF TOKENS) FOR EACH FORKLIFT BACKEND RUNNING AT $\tau = \infty$ AND THE SINGLE-PASS (**SPP**) BASELINE. THE LAST THREE ROWS SUMMARIZE THE TOTAL TOKENS AND TOTAL API COST (IN USD) SUMMED ACROSS ALL EIGHT BENCHMARKS.

Benchmark	**Opus**	**Sonnet**	**GPT-5**	**SPP**
libcsv	733	728	965	39
cjson (parse)	938	919	1194	65
cjson (write)	1038	924	1158	66
libbmp (parse)	409	398	580	18
libbmp (write)	609	617	893	30
pagerank	121	119	163	9
skiplist	622	627	537	32
urlparser	837	819	1085	48
Total tokens (K)	5309	5154	6579	310
Total cost (USD)	\$39.19	\$20.88	\$22.16	\$2.01
Avg cost / bench (USD)	\$4.90	\$2.61	\$2.77	\$0.25

VI. CASE STUDIES

In this section, we present three case studies that further illustrate the reasons behind Forklift's effectiveness.

1) Libcsv - Incremental Lifting vs. Single Pass Baseline: We illustrate the high-level transformations performed by Forklift's passes and describe why they lead to improved idiomaticity for the *libcsv* codebase. Pass 1 removes the custom `realloc` callback from the `csv_parser struct` since the C++ intermediate no longer needs a hand-rolled `grow` function. Pass 2 converts the parser's integer-typed status flags to `bool`. Pass 3 lifts the `char *` byte buffer that `csv_parse` fills one byte at a time to `Vec<u8>`, replaces the manual write-cursor pattern with the vector's built-in `push` method, and deletes the `entry_pos` and `entry_size` fields that the C source carried only to track the cursor and the buffer's capacity. Unlike the single-pass output in Figure 1, which retains `entry_buf` as a raw `*mut u8` and grows it through an `unsafe realloc_fn` helper with per-byte raw-pointer writes, Forklift's final Rust in Figure 4 represents the same buffer as a `Vec<u8>` and appends via the built-in `push` method. As a result, Forklift eliminates the manual `entry_pos` and `entry_size` cursor fields, the hand-rolled buffer-growth helper, and all raw-pointer arithmetic at the write site.

Moreover, the single-pass output keeps `entry_buf` as a raw `*mut u8` and writes to it using pointer arithmetic, which does not have any bounds checks. Such write patterns can be exploited by an attacker and can result in a buffer overflow vulnerability. Forklift's `Vec<u8>` is bounds-checked and safe by construction, so this risk disappears. The multi-pass lifting does not cost performance. The Claude-Sonnet-4.6 $\tau = \infty$ final Rust runs in 6.0 s on *libcsv*, faster than both the original C baseline (8.2 s) and the single-pass LLM-based pipeline's Rust output (7.0 s).

2) SkipList - Performance Threshold Impact: The performance threshold lets a user trade safety for speed, and the

Forklift transpiled Rust code

```
1. pub struct CsvParser {
2.     //remove unused pos,
3.     //size and realloc fields
4.     pub entry_buf: Vec<u8>,
5. }
6. pub unsafe fn csv_parse(
7.     p: &mut CsvParser,
8.     //...more parameters
9. ) -> size_t {
10.    //...more codes
11.    p.entry_buf.push(c);
12.    // safe push, Vec auto grows
13. }
```

Fig. 4. Forklift's Rust output for libcsv

TABLE X
DIFFERENCES BETWEEN THE $\tau = 5\%$ (FASTER) AND $\tau = \infty$ (SAFER) FINAL RUST OUTPUT FOR SKIPLIST. THE **PAS** COLUMN REPORTS THE FORKLIFT LIFTING PASS RESPONSIBLE FOR EACH DIFFERENCE.

#	Difference	τ=5%	τ=∞	Pas
1	`forward` field	`*mut Link`	`Vec<Link>`	3
2	`forward` access	`forward.add(i)`	`forward[i]`	3
3	`forward` alloc	`malloc`	`Vec::with_capacity`	3
4	`head` field	`*mut SkipNodeT`	`Box<SkipNodeT>`	8
5	fn parameter	`*mut SkipListT`	`&mut SkipListT`	8
6	node alloc	`malloc(SkipNodeT)`	`Box::new`	8

skiplist benchmark makes this trade-off concrete. At the strict $\tau = 5\%$ threshold, Forklift produces Rust that runs in 7.8 s, matching the C baseline, while the $\tau = \infty$ threshold runs in 8.6 s, about 10% slower. The slower $\tau = \infty$ output is the safer one. It reduces raw pointers from 39 to 20, total unsafe usages from 224 to 73, and raises the standard-library type usage rate from 14.0% to 43.5%.

Table X traces this gap to the language features that each threshold keeps or rolls back. The $\tau = \infty$ configuration's output adopts two idiomatic constructs that the $\tau = 5\%$ output discards. Pass 3 replaces every node's raw `forward` pointer array with an owned `Vec`, and Forklift's Pass 8 replaces the raw `head` pointer with an owned `Box` and rewrites the `*mut` function parameters as safe references. These conversions remove most of the raw pointers and lets `jrsl_search`, `jrsl_remove`, and other functions drop their `unsafe` qualifier, which is why the $\tau = \infty$ output wins on every safety metric.

Figure 5 shows why the safer output runs slower. The hot path is the skip-list traversal in `jrsl_search` and `jrsl_insert`, which walk the `forward` links of the visited nodes millions of times. In the $\tau = 5\%$ output on the left, `forward` is a raw pointer array and each hop reads a link through direct pointer arithmetic. In the $\tau = \infty$ output on the right, `forward` is a `Vec`, so every hop pays an extra heap indirection and a bounds check. The performance check attributes this slowdown to Pass 3, and under $\tau = 5\%$ it rolls the `Vec` back to the raw pointer array to keep the speed.

TABLE XI
COMPARISON ON LIBCSV OF PASS-BY-PASS LIFTING FROM UNSAFE RUST TO SAFE RUST AGAINST FORKLIFT. BOTH METHODS RUN AT THE $\tau = \infty$ THRESHOLD. THE COLUMNS REPORT THE NUMBER OF RAW POINTER DEFINITIONS, THE TOTAL UNSAFE USAGE, AND THE RUST RUNTIME IN SECONDS.

Method	ptr	unsafe	performance(s)
Forklift-Rust ($\tau = \infty$)	16	45	9.1
Forklift ($\tau = \infty$)	**13**	**36**	**6.0**

3) Libcsv: Impact of Alternate Intermediate Language: To evaluate the effectiveness of C++ as an intermediate language, we design an alternate pipeline, Forklift-Rust, that routes the transpilation through unsafe Rust instead of C++. Our experiments show that Forklift-Rust has unpredictable performance for each pass, and also generates more unsafe code than Forklift.

To compare multi-pass lifting through C++ with multi-pass lifting through unsafe Rust, we feed C2Rust's [1] unsafe Rust output through the same pass-by-pass lifting pipeline that Forklift applies to C++. Similar to Forklift, each pass in either pipeline focuses on one Rust feature—for example, replacing raw-pointer arithmetic with `Vec` indexing or wrapping a nullable handle in `Option`. Running both pipelines on *libcsv* at the $\tau = \infty$ performance check threshold lets us isolate which intermediate language representation gives the LLM a better starting point for producing safe, idiomatic Rust.

The C++ intermediate language produces the better final Rust on every metric. As Table XI shows, Forklift's Rust output has fewer unsafe raw-pointer usages (13 vs. 16), a lower total unsafe-usage count (36 vs. 45), and a faster runtime (6.0 s vs. 9.1 s) than the Forklift-Rust pipeline's Rust output. The largest gap is on runtime, where Forklift is 3.1 s (34%) faster. Forklift also generates the safer output, using 18.8% fewer raw pointers and 20% fewer total unsafe usages.

A further problem with the C $\rightarrow$ unsafe Rust route is that the runtime shifts before any safety lifting is applied. The original *libcsv* C source runs in 8.2 s. After one C++ lifting pass Forklift's intermediate runs in 8.1 s, essentially matching the C baseline. C2Rust's unsafe Rust output, in contrast, runs in 7.5 s before any safety lifting is applied, an 8.5% discrepancy from the original C that the C $\rightarrow$ unsafe Rust translation introduces for no semantic reason. This shift makes it hard to attribute later per-pass performance changes to the safety lifting itself rather than to the initial C $\rightarrow$ unsafe Rust transition.

VII. DISCUSSION

Adaptive and customizable lifting. The set of lifting passes that Forklift applies today is fixed in advance and chosen heuristically rather than derived from the program under transpilation. These pass boundaries are a design choice and not a fundamental limit, and a single pass can be divided into finer ones whenever doing so isolates an independent decision. Pass 4, for example, currently lifts a C character pointer into either an owned string or a borrowed string view within one

Forklift transpiled Rust code ($\tau = 5\%$)

```rust
pub struct SkipNodeT {
    pub forward: *mut Link,
    //more fields...
}
pub struct SkipListT {
    pub head: *mut SkipNodeT,
    //...more fields...
}
pub unsafe fn jrsl_search(skip_list: *mut SkipListT,
                          key: *mut c_void)
    -> *mut c_void {
    let mut x: *mut SkipNodeT = (*skip_list).head;
    let mut i = (*skip_list).level as usize;
    while i > 0 {
        //...more codes
        x = (*x).forward.add(i - 1).read().node;
        // unsafe pointer arithmetic
    }
}
```

Forklift transpiled Rust code ($\tau = \infty$)

```rust
pub struct SkipNodeT {
    pub forward: Vec<Link>,
    //...more codes
}
pub struct SkipListT {
    pub head: Box<SkipNodeT>,
    //...more codes
}
pub fn jrsl_search(skip_list: &mut SkipListT,
                   key: *mut c_void)
    -> *mut c_void {
    let mut x: *mut SkipNodeT = skip_list.head.as_mut();
    let mut i = skip_list.level as usize;
    while i > 0 {
        //...more codes
        x = (*x).forward[i - 1].node; // safe indexing
    }
}
```

Fig. 5. Side-by-side comparison of the Rust output produced by Forklift of $\tau = 5\%$ (left) and $\tau = \infty$ (right) for skiplist.

pass, and we could split it into two passes where the first introduces owned strings and the second introduces borrowed string views. Forklift is built to accommodate this kind of change, since it already supports reordering passes and adding new customized passes to the pipeline. The natural next step is to make the pipeline adaptive, so that Forklift first analyzes a program and then selects only the passes that program needs instead of running every pass in a fixed order. A program with no hash tables, for instance, would skip the map pass entirely.

Applicability. Because Forklift concentrates on rewriting data types, it helps most on programs that lean heavily on complex data structures and least on those that do not. A program dominated by scalar arithmetic over primitive data type exposes little for the lifting passes to act on, so its gains over a single-pass translation are small.

VIII. Related Work

C-to-Rust Transpilation. Manual rewriting of mature C codebases to Rust is labor intensive, which has motivated a large body of work on automated transpilation. Various analysis-based techniques [13], [14], [27], [48] transpile C to Rust by refining the intermediate unsafe Rust output of the C2Rust tool [1]. These approaches guarantee the correctness of the translation but often produce unidiomatic Rust code [15]. Targeted static analysis tools convert specific constructs to safe Rust, replacing C lock APIs [21], pointer-based output parameters [22], or both [19], recovering generics from void pointers [45], translating C-to-Rust interfaces [9], and handling formally verified C [16]. Cpp2Rust [38] targets C++ rather than C, using a syntax-directed translator that turns C++ into memory-safe Rust by inserting runtime ownership and mutability checks. Various approaches combine program analysis techniques with LLM-based transpilation, such as ownership analysis [50], type analysis [20], code slicing [8], code segmentation [40], and control-flow and data-flow analysis [32]. Others pair the LLM with peephole pointer rewriting [17], translation rules [47], context preprocessing [6], staged prompting and repair [49], semantics augmented retrieval [29], or skeleton guided project level translation [43]. The benchmark suites [24], [35] evaluate the effectiveness of these C-to-Rust transpilers. All of these methods ask the LLM to cross every Rust feature in a single pass. Forklift instead lifts C through a C++ intermediate pass by pass with one feature per pass, which lets it reshape data structures and ownership incrementally and measure the performance cost of each lift.

Performance of LLM-Generated Code. Empirical studies report that LLM-generated code is often functionally correct yet slower than human-written code, with root causes such as inefficient function calls, loops, algorithms, and language-feature use [25], [26]. Evaluations of many LLMs on Leetcode show the gap is not universal and that generated code can even be more efficient than human solutions on average [11], [44]. EffiBench-X [39] provides the first multi-language efficiency benchmark, and PERFCODEGEN [37] raises efficiency with runtime feedback from test execution. These works target code an LLM generates within a single language. Forklift instead pursues safety and performance together, measuring the runtime cost of each transpilation pass and discarding any pass that exceeds a configurable threshold, so the Rust it generates is both safe and performant.

IX. Conclusion

We presented Forklift, an LLM-based pipeline that transpiles C to idiomatic Rust by lifting it incrementally through C++ rather than in a single pass. Across eight benchmarks from six C codebases and three LLM backends, Forklift produced substantially safer and more idiomatic Rust than a single-pass baseline at comparable runtime. With Claude-Sonnet-4.6 as the backend and the same model driving the single-pass baseline, Forklift raised the standard-library type usage rate by $4.5\times$, reduced total unsafe usages by 82.2% and raw pointers by 73.1%, while matching or beating its performance on five of the eight benchmarks.

X. Data Availability Statement

To prevent premature disclosure, our source code and dataset are provided for review via a private link (https://figshare.com/s/dc0aadb31cea79ba905d). Upon acceptance, the camera-ready revision will be updated to include a permanent, public DOI.

References

[1] "C2rust," 2024. [Online]. Available: https://github.com/immunant/c2rust

[2] "Unsafe Rust, The Rust Programming Language," 2024. [Online]. Available: https://doc.rust-lang.org/book/ch20-01-unsafe-rust.html

[3] "AST Matcher Reference," https://clang.llvm.org/docs/LibASTMatchersReference.html, 2025.

[4] Anthropic, "Introducing Claude Opus 4.6," https://www.anthropic.com/news/claude-opus-4-6, 2026, accessed: 2026-06-03.

[5] ——, "Introducing Claude Sonnet 4.6," https://www.anthropic.com/news/claude-sonnet-4-6, 2026, accessed: 2026-06-25.

[6] Y. Bai and T. Palit, "Rustassure: Differential symbolic testing for llm-transpiled c-to-rust code," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 534–546.

[7] M. Brunsfeld *et al.*, "tree-sitter: An incremental parsing system for programming tools," https://tree-sitter.github.io/tree-sitter/, 2018.

[8] X. Cai, J. Liu, X. Huang, Y. Yu, H. Wu, C. Li, B. Wang, I. N. B. Yusuf, and L. Jiang, "Rustmap: Towards project-scale c-to-rust migration via program analysis and llm," *arXiv preprint arXiv:2503.17741*, 2025.

[9] V. Chen, A. Coughlin, and M. D. Bond, "&inator: Correct, precise C-to-Rust interface translation," *Proceedings of the ACM on Programming Languages*, vol. 10, no. PLDI, 2026.

[10] J. Coe, A. Peacock, and S. Parent, "Vocabulary types for composite class design," https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p3042r0.pdf, Mar. 2023, accessed 2026-03-24.

[11] T. Coignion, C. Quinton, and R. Rouvoy, "A performance study of LLM-generated code on leetcode," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2024, pp. 79–89.

[12] P. Deligiannis, A. Lal, N. Mehrotra, and A. Rastogi, "Fixing rust compilation errors using llms," *arXiv preprint arXiv:2308.05177*, 2023.

[13] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf, "Aliasing limits on translating c to safe rust," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 551–579, 2023.

[14] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, "Translating c to safer rust," *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–29, 2021.

[15] H. F. Eniser, H. Zhang, C. David, M. Wang, M. Christakis, B. Paulsen, J. Dodds, and D. Kroening, "Towards translating real-world code with llms: A study of translating to rust," *arXiv preprint arXiv:2405.11514*, 2024.

[16] A. Fromherz and J. Protzenko, "Compiling c to safe rust, formalized," 2024. [Online]. Available: https://arxiv.org/abs/2412.15042

[17] Y. Gao, C. Wang, P. Huang, X. Liu, M. Zheng, and X. Zhang, "Pr2: Peephole raw pointer rewriting with llms for translating c to safer rust," *arXiv preprint arXiv:2505.04852*, 2025.

[18] Google, "Secure by design: Google's perspective on memory safety," https://security.googleblog.com/2024/03/secure-by-design-googles-perspective-on.html, Mar. 2024, accessed 2026-03-24.

[19] J. Hong, "Improving automatic c-to-rust translation with static analysis. in 2023 ieee/acm 45th international conference on software engineering: Companion proceedings (icse-companion)," 2023.

[20] J. Hong and S. Ryu, "Type-migrating c-to-rust translation using a large language model," vol. 30, no. 1, p. 3, publisher: Springer.

[21] ——, "Concrat: An automatic c-to-rust lock api translator for concurrent programs," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 716–728.

[22] ——, "Don't write, but return: Replacing output parameters with algebraic data types in c-to-rust translation," *Proceedings of the ACM on Programming Languages*, vol. 8, no. PLDI, pp. 716–740, 2024.

[23] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "Safe systems programming in rust," *Communications of the ACM*, vol. 64, no. 4, pp. 144–152, 2021.

[24] A. Khatry, R. Zhang, J. Pan, Z. Wang, Q. Chen, G. Durrett, and I. Dillig, "Crust-bench: A comprehensive benchmark for c-to-safe-rust transpilation," 2025. [Online]. Available: https://arxiv.org/abs/2504.15254

[25] S. Li, Y. Cheng, J. Chen, J. Xuan, S. He, and W. Shang, "Assessing the performance of ai-generated code: A case study on github copilot," in *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2024, pp. 216–227.

[26] ——, "Performance analysis of ai-generated code: A case study of copilot, copilot chat, codellama, and deepseek-coder models," *Empirical Software Engineering*, vol. 31, no. 3, p. 62, 2026.

[27] M. Ling, Y. Yu, H. Wu, Y. Wang, J. R. Cordy, and A. E. Hassan, "In rust we trust: a transpiler from unsafe c to safer rust," in *Proceedings of the ACM/IEEE 44th international conference on software engineering: companion proceedings*, pp. 354–355.

[28] I. Lozano and D. Maier, "Deploying rust in existing firmware codebases," https://security.googleblog.com/2024/09/deploying-rust-in-existing-firmware.html, Sep. 2024, google Security Blog.

[29] F. Luo *et al.*, "Integrating rules and semantics for LLM-based C-to-Rust translation," in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME), Industry Track*, 2025, arXiv:2508.06926.

[30] N. D. Matsakis and F. S. Klock, "The rust language," in *Proceedings of the 2014 ACM SIGAda annual conference on High integrity language technology*, 2014, pp. 103–104.

[31] Mozilla, "Put your trust in rust — shipping now in firefox," https://blog.mozilla.org/en/firefox/put-trust-rust-shipping-now-firefox/, Mar. 2017, accessed 2026-03-24.

[32] V. Nitin, R. Krishna, L. L. d. Valle, and B. Ray, "C2saferrust: Transforming c projects into safer rust with neurosymbolic techniques," *arXiv preprint arXiv:2501.14257*, 2025.

[33] A. Nunez, N. T. Islam, S. K. Jha, and P. Najafirad, "Autosafecoder: A multi-agent framework for securing llm code generation through static analysis and fuzz testing," *arXiv preprint arXiv:2409.10737*, 2024.

[34] OpenAI, "Introducing GPT-5.4," https://openai.com/index/introducing-gpt-5-4/, 2026, accessed: 2026-06-25.

[35] G. Ou, M. Liu, Y. Chen, X. Peng, and Z. Zheng, "Repository-level code translation benchmark targeting rust," 2025. [Online]. Available: https://arxiv.org/abs/2411.13990

[36] R. Pan, A. R. Ibrahimzada, R. Krishna, D. Sankar, L. P. Wassi, M. Merler, B. Sobolev, R. Pavuluri, S. Sinha, and R. Jabbarvand, "Lost in translation: A study of bugs introduced by large language models while translating code," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[37] Y. Peng, A. D. Gotmare, M. R. Lyu, C. Xiong, S. Savarese, and D. Sahoo, "Perfcodegen: Improving performance of llm generated code with execution feedback," in *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, 2025, pp. 1–13.

[38] L. Popescu, F. Gouveia, H. Preto, J. Silveira, D. Hrybenko, J. Fragoso Santos, and N. P. Lopes, "Cpp2Rust: Automatic translation of C++ to safe rust," *Proceedings of the ACM on Programming Languages*, vol. 10, no. PLDI, 2026.

[39] Y. Qing, B. Zhu, M. Du, Z. Guo, T. Y. Zhuo, Q. Zhang, J. M. Zhang, H. Cui, S.-M. Yiu, D. Huang, S.-K. Ng, and L. A. Tuan, "EffiBench-X: A multi-language benchmark for measuring efficiency of LLM-generated code," *arXiv preprint arXiv:2505.13004*, 2025.

[40] M. Shiraishi and T. Shinagawa, "Context-aware code segmentation for c-to-rust translation using large language models," 2024. [Online]. Available: https://arxiv.org/abs/2409.10506

[41] The Linux Kernel Documentation, "Rust," https://docs.kernel.org/rust/index.html, 2026, accessed 2026-03-24.

[42] The LLVM Project, "The LLVM compiler infrastructure project," https://llvm.org/, 2026, accessed: 2026-06-30.

[43] C. Wang, T. Yu, B. Shen, J. Wang, D. Chen, W. Zhang, Y. Shi, C. Xie, and X. Gu, "EvoC2Rust: A skeleton-guided framework for project-level C-to-Rust translation," *arXiv preprint arXiv:2508.04295*, 2025.

[44] L. Wang, C. Shi, S. Du, Y. Tao, Y. Shen, H. Zheng, and X. Qiu, "Performance review on llm for solving leetcode problems," in *2024 4th International Symposium on Artificial Intelligence and Intelligent Manufacturing (AIIM)*. IEEE, 2024, pp. 1050–1054.

[45] X. Wu and B. Demsky, " GenC2Rust: Towards Generating Generic Rust Code from C ," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 664–664. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00127

[46] A. Z. Yang, Y. Takashima, B. Paulsen, J. Dodds, and D. Kroening, "Vert: Verified equivalent rust transpilation with large language models as few-shot learners," *arXiv preprint arXiv:2404.18852*, 2024.

[47] H. Zhang, C. David, M. Wang, B. Paulsen, and D. Kroening, "Scalable, validated code translation of entire projects using large language models," 2024. [Online]. Available: https://arxiv.org/abs/2412.08035

[48] H. Zhang, C. David, Y. Yu, and M. Wang, "Ownership guided c to rust translation," in *International Conference on Computer Aided Verification*. Springer, pp. 459–482.

[49] R. Zhang, S. Zhang, and L. Xie, "A systematic exploration of C-to-Rust code translation based on large language models: prompt strategies and automated repair," *Automated Software Engineering*, vol. 33, 2025.

[50] T. Zhou, H. Lin, S. Jha, M. Christodorescu, K. Levchenko, and V. Chandrasekaran, "Llm-driven multi-step translation from c to rust using static analysis," *arXiv preprint arXiv:2503.12511*, 2025.